

Ruprecht-Karls-Universität Heidelberg
Institut für Informatik

Bachelor Thesis
Parallel ASP Planning

Name: Levi Maximilian Klein
Student ID: 4283957
Supervisor: Dr. Gnad
Submission date: 17.02.2026

Ich versichere, dass ich diese Bachelor-Arbeit selbstständig verfasst und nur die angegebenen Quellen und Hilfsmittel verwendet habe.

Heidelberg, dd.mm.yyyy

Abstract

This is the abstract.

Contents

List of Figures	1
1 Introduction	3
1.1 Motivation	3
1.2 Goal of the Thesis	3
1.3 Structure of the Thesis	3
2 Theory	4
2.1 Introduction to Planning Task and Parallel Planning	4
2.2 Introduction to ASP	8
2.2.1 Extension for sequential plans	12
2.2.2 Extension for $\forall$ -step plans	12
2.2.3 encoding for $\exists$ -step plans	14
2.2.4 encoding for relaxed $\exists$ -step plans	15
2.3 Proof of Correctness	16
3 Method	24
3.1 Pipeline Overview	24
3.2 Changes in the ASP encodings	24
3.3 Optimizations	25
3.3.1 Mutexes	25
3.3.2 Guess and Check	26
3.3.3 Heuristic	27
3.4 Experiment	28
4 Results	29
4.1 Overall Performance	29
4.1.1 Coverage Analysis	29
4.1.2 Runtime Analysis	30
Bibliography	32

List of Figures

3.1	Pipeline for testing the different planning encodings	24
4.1	Coverage analysis: Number of solved instances per algorithm across benchmark domains. The heatmap indicates the absolute number of solved instances (left) out of the total available instances (right).	30
4.2	Runtime analysis: Average solving time (in seconds) for successfully solved instances. Red cells indicate higher runtimes, while light yellow cells indicate faster performance.	31

List of Listings

1	ASP fact representation of the example planning task	10
2	Core for sequential and parallel encodings for planning	11
3	Extension of Listing 2 for encoding sequential plans	12
4	Output from clingo for sequential encoding	12
5	Extension of Listing 2 for encoding $\forall$ -step plans	12
6	Output from clingo for $\forall$ -step encoding	13
7	Extension for $\exists$ -step plans	14
8	Alternative Extension for $\exists$ -step plans	14
9	Output from clingo for $\exists$ -step encoding	15
10	Extension for relaxed $\exists$ -step plans	15
11	Output from clingo for relaxed $\exists$ -step encoding	16
12	Expansion for mutexes	26
13	Encoding for checking circular invalidation	27
14	backward propagating heuristic	28

1 Introduction

Automated planning is a fundamental area of Artificial Intelligence that deals with finding a sequence of actions to transition a system from an initial state to a desired goal state. Finding an efficient and effective plan is crucial in various applications and industries such as robotics and logistics. In general, planning problems are computationally challenging, often falling into NP-hard or even PSPACE-complete complexity classes. Therefore, developing efficient algorithms and representations for planning tasks to reduce the search space and computation time is of great importance.

One of the strengths of Classical Planning is its domain independence. This means planning techniques can be used in many different fields without adjusting the core methods. This makes classical planning highly flexible and widely applicable.

1.1 Motivation

1.2 Goal of the Thesis

1.3 Structure of the Thesis

Here you describe the structure of the thesis. For example: In chapter [2](#), we formally define the notion of planning tasks, first describing sequential plans and afterward three different kinds of parallel plans. We then present the ASP encodings for the different plans and proofing their soundness. In chapter [3.1](#) we show the pipeline from translating a pddl problem to validating the given plan and introduce different optimization strategies

2 Theory

2.1 Introduction to Planning Task and Parallel Planning

Before discussing the specific encodings and optimizations for parallel planning, we establish a precise mathematical model of the planning problem.

Definition 1 *Multi-valued planning task*. A STRIPS-like multivalued planning task according to (Helmert 2006) is given by a 4 tuple $\langle \mathcal{F}, s_0, s_*, \mathcal{O} \rangle$, where:

- $\mathcal{F}$ is a finite set of state variables, also called **fluents**. Each $x \in \mathcal{F}$ is associated with a finite domain $\text{dom}(x)$. In any given state, a fluent x is assigned exactly one value $v \in \text{dom}(x)$.
- s_0 , the **initial state**, is a total function, s.t. $s_0(x) \in \text{dom}(x)$ for each $x \in \mathcal{F}$, i.e. a function that maps each fluent to a value from its domain.
- s_* , the **goal conditions**, is a partial function, s.t. $s_*(x) \in \text{dom}(x)$, for each $x \in \text{dom}(s_*)$ i.e. a function that specifies the desired values for a subset of fluents.
- $\mathcal{O}$ is a finite set of operators, also called **actions**. An action $a \in \mathcal{O}$ is a tuple $\langle a^c, a^e \rangle$, where a^c is the *precondition* and a^e is the *postcondition* of a . Both a^c and a^e are partial functions, such that $a^c(x) \in \text{dom}(x)$ for each $x \in \text{dom}(a^c)$ and $a^e(x) \in \text{dom}(x)$ for each $x \in \text{dom}(a^e)$.

Remark. In many implementations, actions also include **conditional effects** $\text{eff} = \langle \text{cond}, x, v \rangle$, where the fluent $x \in \mathcal{F}$ is assigned value $v \in \text{dom}(x)$ iff cond , a partial function called the *condition* such that $\text{cond}(x) \in \text{dom}(x)$ for each $x \in \text{dom}(\text{cond})$, holds in the current state. However, for the definitions and ASP encodings discussed in this thesis, we adopt a simplified view where effect conditions are implicitly assumed to be satisfied (i.e., mapping any effect on x to $a^e(x)$ regardless of its condition). Thus, a^e denotes the union of all variable assignments an action can potentially trigger.

Notation. Formally, the fact that a fluent X is assigned the value V in a state s is denoted by $s(X) = V$. For readability, we use the compact notation $X = V$ to denote this assignment when the reference to the specific state s is implicit from the context.

Example 1. Let us illustrate the definitions with a small example. The plan is to leave the house by opening the door, going outside, and finally closing the door.

- $\mathcal{F} = \{\text{door}, \text{location}\}$ with $\text{dom}(\text{door}) = \{\text{open}, \text{closed}\}$, $\text{dom}(\text{location}) = \{\text{inside}, \text{outside}\}$

- Initial state: $s_0 = \{door=closed, location=inside\}$
- Goal: $s_* = \{door=closed, location=outside\}$
- Actions: $\mathcal{O} = \{openDoor, closeDoor, goOut\}$, where

$$openDoor = \langle \{door=closed\}, \{door=open\} \rangle$$

$$closeDoor = \langle \{door=open\}, \{door=closed\} \rangle$$

$$goOut = \langle \{location=inside, door=open\}, \{location=outside\} \rangle$$

Intuitively, fluents can be understood as the dynamic properties that describe the configuration of the world. In our running example, this world is characterized by the *location* of the actor and the status of the *door*. The initial state captures the complete snapshot of the world at the beginning. This stands in contrast to the goal conditions, which represent the desired outcome of the planning task. Unlike the initial state, the goal is typically partial, meaning we only specify the values we explicitly care about, leaving other variables undefined. Finally, actions define the transition logic of the system: they describe how the world changes, where the precondition establishes the necessary requirements for the action to be applicable, and the postcondition specifies the resulting effects.

With these concepts in place, we now formally define the state transition function, which dictates how applying an action transforms one state into another.

Definition 2 Successor state. Let s be state and $a \in \mathcal{O}$ an action.

The *successor state* $o(a, s)$, obtained by applying action a in state s , is defined if the precondition of a holds, i.e., $a^c(x) = s(x)$ for all $x \in \text{dom}(a^c)$. Then

$$s'(x) = o(a, s) = \begin{cases} a^e(x) & \text{if } x \in \text{dom}(a^e) \\ s(x) & \text{otherwise.} \end{cases}$$

Remark. Let $A_n = \langle a_1, \dots, a_n \rangle$ be a sequence of actions with length n . We define the application of A_n to a state s recursively by

$$o(A_n, s) = o(a_n, o(A_{n-1}, s))$$

if $o(A_i, s)$ is defined for all $1 \leq i \leq n$.

We can represent these transitions in a directed graph, where each node correspond to a distinct state, while the edges represent the possible actions from one state into another. Consequently, a solution equates to finding a path through this graph. Graphically, this transition mechanism induces a directed graph structure known as the *state space*. In this representation, the nodes correspond to the distinct states of the world, while the edges represent the actions transforming one state into another. Consequently, finding a solution equates to identifying a path through this graph. Since this graph grows exponentially with the number

of actions, efficiently searching through this state space is a central challenge in automated planning.

Definition 3 *Sequential plan.* A *sequential plan* is a sequence of actions $A_n = \langle a_1, \dots, a_n \rangle$, s.t. $s' = o(A_n, s_0)$ is defined and $s'(x) = s_*(x)$ for all $x \in \text{dom}(s_*)$.

Example 2. Consider the planning task from the previous example. One possible sequential plan is $A = \langle \text{openDoor}, \text{goOut}, \text{closeDoor} \rangle$. Since

$$\begin{aligned} o(A, s_0) &= o(\langle \text{openDoor}, \text{goOut}, \text{closeDoor} \rangle, \{\text{door}=\text{closed}, \text{location}=\text{inside}\}) \\ &= o(\langle \text{goOut}, \text{closeDoor} \rangle, \{\text{door}=\text{open}, \text{location}=\text{inside}\}) \\ &= o(\text{closeDoor}, \{\text{door}=\text{open}, \text{location}=\text{outside}\}) \\ &= \{\text{door}=\text{closed}, \text{location}=\text{outside}\} = s_* \end{aligned}$$

While the sequential approach is intuitive, it is often inefficient searching through the whole state space. A possible way to reduce the horizon required to find a solution is to allow multiple actions to be executed simultaneously, forming a parallel plan. However, arbitrary sets of actions cannot simply be executed simultaneously, since conflicts may arise if actions interfere with each other. The most fundamental requirement for a set of actions to be valid is that they must agree on their postcondition. If two actions executing in parallel attempt to assign different values to the same variable, the resulting state would be undefined. To prevent this ambiguity, we introduce the concept of *confluence*.

Definition 4 *confluent.* Let $A = \{a_1, \dots, a_n\} \subseteq \mathcal{O}$ be a set of actions.

A is *confluent* : $\iff a_i^e(x) = a_j^e(x)$ for all $1 \leq i < j \leq n$ and each $x \in \text{dom}(a_i^e) \cap \text{dom}(a_j^e)$

Remark. A is confluent $\iff B$ is confluent for all $B \subseteq A$

Example 3. In our initial example, the set $A = \{\text{openDoor}, \text{goOut}\}$ is confluent because the domains of their effects are disjoint. Similarly, $\{\text{closeDoor}, \text{goOut}\}$ is confluent.

To illustrate more complex scenarios, we extend our example with two fluents, *light* and *key*, where $\text{dom}(\text{light}) = \text{dom}(\text{key}) = \{0, 1\}$, and two corresponding actions:

$$\begin{aligned} \text{lightsOut} &= \langle \{\text{location}=\text{inside}, \text{light}=1\}, \{\text{light}=0\} \rangle \\ \text{getKey} &= \langle \{\text{location}=\text{inside}, \text{key}=0\}, \{\text{key}=1\} \rangle \end{aligned}$$

Additionally, we modify the precondition of *openDoor* to require the key ($\text{openDoor}^c = \{\text{door}=\text{closed}, \text{key}=1\}$) and update the goal to $s_* = \{\text{location}=\text{outside}, \text{door}=\text{closed}, \text{light}=0\}$.

In this extended setting, the set $A' = \{\text{getKey}, \text{openDoor}, \text{goOut}, \text{lightsOut}\}$ is also a confluent set of actions. Even though the actions are semantically related, their effects target different variables (*key*, *door*, *location*, *light*), so no conflicts in their effects occur.

The concept of confluence ensures that the simultaneous execution of actions within a set A does not lead to contradictory state updates regarding the postcondition of the actions $a \in A$. However, ensuring consistent effects is only one part of the problem, since we also need to consider the preconditions of the actions. Unlike in sequential planning, where preconditions are simply checked against the current state, parallel planning allows for different interpretations of precondition dependencies. These interpretations lead to the different parallel representations of sequential plans, which have been investigated in the literature (Dimopoulos, Nebel, and Koehler 1997); (Rintanen, Heljanko, and Niemelä 2006); (Wehrle and Rintanen 2007) and are specifically implemented and tested using Answer Set Programming in (DIMOPOULOS et al. 2019).

Definition 5 *Parallel representations*. Let s be a state and $A = \{a_1, \dots, a_n\}$ be a confluent set of actions. Then A is called:

- (i) $\forall$ -step serializable in s $:\iff o(\langle a_{\pi(1)}, \dots, a_{\pi(n)} \rangle, s)$ is defined for **all** permutations π
- (ii) $\exists$ -step serializable in s $:\iff o(\langle a_{\pi(1)}, \dots, a_{\pi(n)} \rangle, s)$ is defined for **some** permutation π and $a^c(x) = s(x)$ for each $a \in A, x \in \text{dom}(a^c)$
- (iii) relaxed $\exists$ -step serializable in s $:\iff o(\langle a_{\pi(1)}, \dots, a_{\pi(n)} \rangle, s)$ is defined for **some** permutation π

Remark. A is $\forall$ -step serializable $\Rightarrow A$ is $\exists$ -step serializable $\Rightarrow A$ is relaxed $\exists$ -step serializable.

Each parallel representation has different restrictions on how the preconditions of the actions in A are satisfied with $\forall$ -step being the most restrictive and relaxed $\exists$ -step being the least restrictive.

- **$\forall$ -step serialization** requires strong independence: Since the set must be executable in *any* order, no action is allowed to invalidate the preconditions of another action in the same set and since any action $a \in A$ must be executable at the start, all preconditions have to be satisfied in the current state s .
- **$\exists$ -step serialization** relaxes the independence. Preconditions must still be satisfied in s (i.e., actions cannot enable each other). However, conflicts are allowed: One action $a \in A$ may destroy the precondition of another action $a' \in A$, provided that there exists *some* valid execution order where a comes after a' .
- **Relaxed $\exists$ -step serialization** is the least restrictive. It drops the requirement that preconditions must hold in s . An action $a \in A$ can establish the preconditions required by another action $a' \in A$, as long as they can be ordered such that the a comes before a' .

Based on these definitions of valid parallel steps, we can now define the corresponding parallel

plans.

Definition 6 *Parallel Plans*. Let $A = \langle A_1, \dots, A_m \rangle$ be a sequence of confluent sets of actions. A is a $\forall$ -step, $\exists$ -step or relaxed $\exists$ -step plan iff

- $s_m(x) = s_*(x)$ for each $x \in \text{dom}(s_*)$
- A_i is $\forall$ -step, $\exists$ -step or relaxed $\exists$ -step serializable in s_{i-1} for $1 \leq i \leq m$, where

$$s_i(x) = \begin{cases} a^e(x) & \text{if } x \in \text{dom}(a^e) \\ s_{i-1}(x) & \text{otherwise} \end{cases}$$

for each $a \in A_i$.

Example 4. In our modified example we have:

- $\forall$ -step plan: $\langle \{getKey, lightsOut\}, \{openDoor\}, \{goOut\}, \{closeDoor\} \rangle$
- $\exists$ -step plan: $\langle \{getKey, lightsOut\}, \{openDoor\}, \{goOut, closeDoor\} \rangle$
- relaxed $\exists$ -step plan: $\langle \{getKey, lightsOut, openDoor, goOut\}, \{closeDoor\} \rangle$

While the concept of a sequential plan is straightforward, we are effectively dealing with transitions of the form $s \xrightarrow{a} s'$. In parallel planning, however, we generalize this transition to allow sets of actions, i.e., $s \xrightarrow{A} s'$, effectively merging multiple actions together into a single one.

Summary

In this section, we have established the formal framework for multi-valued planning tasks. We defined the static components of the world (fluents and states), the dynamic transition logic (applying actions), and the conditions under which sets of actions can be executed in parallel. With these formal definitions in place, the next chapter will demonstrate how to translate this theoretical framework into concrete Answer Set Programming encodings to solve planning tasks efficiently.

2.2 Introduction to ASP

Answer Set Programming (ASP) is a declarative problem-solving approach oriented towards difficult (primarily NP-hard) search problems. Unlike imperative programming, where the programmer specifies the control flow to compute a solution, ASP follows a declarative paradigm: the problem is described as a logic program such that its valid models—called *answer sets*—correspond to the solutions of the original problem. This methodology allows for a clean separation between the problem description (the logic program) and the solving algorithms (implemented by ASP systems like *clingo*).

The theoretical foundation of ASP lies in the stable model semantics of logic programming (Gelfond and Lifschitz 1988). To formally define these programs, we first introduce their basic

building blocks. The most fundamental units are *atoms* of the form $p(t_1, \dots, t_n)$ or p , which can either be true or false. These atoms are combined to form literals. A literal is defined as either an atom A (a positive literal) or its default negation *not* A (a negative literal). Here, *not* denotes negation as failure, meaning that *not* A is considered true if no proof for A can be derived from the program.

Logic Programs

The head of a rule or fact is an atom, which has the same syntactic form as a constant or a function, i.e. $prec(X, V, t)$. The body consists of a (possible empty) list of literals $L_1, \dots, L_n$. Each L_i is of the form A or *not* A , where A is an atom and *not* the default negation. The “:-” operator can be read as “ $\Leftarrow$ ”, therefore a rule can be interpreted as “if all literals in the body are true, then the head is true”. A fact is a rule without a body, meaning that the head atom is unconditionally true. An integrity constraint is a rule without a head, that filters solution candidates, meaning that all the literal in its body must not be satisfied, i.e. at least one literal in the body must be false. A set of atoms is called a *model* of a logic program, if all facts, rules and integrity constraints are satisfied. Atoms are considered true if they are in the model, otherwise they are considered false. In ASP, a model is called an *answerset* or *stablemodel*, if every atom in the model has an acyclic derivation of the program.

Based on these definitions, we can specify the structure of the programs used to model planning tasks. A *normal logic program* consists of a finite set of rules, facts, and integrity constraints. These constructs generally follow the form:

$$\begin{aligned} \textbf{Fact:} \quad & A_0. \\ \textbf{Rule:} \quad & A_0 :- L_1, \dots, L_n. \\ \textbf{Constraint:} \quad & :- L_1, \dots, L_n. \end{aligned}$$

The *head* of a rule is an atom A_0 . The *body* consists of a (possibly empty) conjunction of literals $L_1, \dots, L_n$. The symbol “:-” stands for logical implication ($\Leftarrow$) and allows the rule to be interpreted as: “if all literals in the body are true, then the head must be true”.

We distinguish three types of rules:

- A **fact** is a rule with an empty body, implying that the head atom A_0 is unconditionally true.
- A **rule** derives the head A_0 if the body is satisfied.
- An **integrity constraint** is a rule with an empty head. It serves as a filter for solution candidates: At least one literal in the body must be false, otherwise the candidate model is rejected.

A set of atoms is called a model of a logic program if it satisfies all rules, facts, and integrity constraints. Atoms are considered true iff they are contained in the model. In ASP, a model is

called an answer set if every atom in the model has an acyclic derivation from the program. Based on these semantics, we can now translate the formal definition of a planning task into an ASP logic program. The problem instance is represented purely by facts: the initial state s_0 is defined via `init/2`, the goal s_* via `goal/2`, and the pre- and postconditions of actions via `prec/3` and `post/3` respectively. Listing 1 shows the complete fact representation for our running example.

```
1  fluent(door; loc; light; key).
2  value(door, 0).  value(door, 1).
3  value(loc, in).  value(loc, out).
4  value(light, 0). value(light, 1).
5  value(key, 0).   value(key, 1).
6
7  init(door, 0). init(loc, in).  init(light, 1). init(key, 0).
8  goal(door, 0). goal(loc, out). goal(light, 0).
9
10 action(openDoor; closeDoor; goOut; lightOut; getKey).
11 prec(openDoor, door, 0).  prec(openDoor, key, 1).  post(openDoor, door, 1).
12 prec(closeDoor, door, 1).                                post(closeDoor, door, 0).
13 prec(goOut, loc, in).    prec(goOut, door, 1).  post(goOut, loc, out).
14 prec(lightOut, loc, in).  prec(lightOut, light, 1). post(lightOut, light, 0).
15 prec(getKey, loc, in).    prec(getKey, key, 0).   post(getKey, key, 1).
```

Listing 1: ASP fact representation of the example planning task

Combining these facts with the right encodings, stable models correspond to the *sequential*, $\forall$ -*step*, $\exists$ -*step* or *relaxed $\exists$ -step plans*.

```

1  #include <incmode>.
2  holds(X, V, 0) :- init(X, V).
3  #program check(t).
4  :- query(t), goal(X, V), not holds(X, V, t).
5  #program step(t).
6  {holds(X, V, t) : value(X, V)} = 1 :- fluent(X).
7  {occurs(A, t)} :- action(A).
8  :- occurs(A, t), post(A, X, V), not holds(X, V, t).
9  :- holds(X, V, t), not holds(X, V, t-1), not occurs(A, t) : post(A, X, V).

```

Listing 2: Core for sequential and parallel encodings for planning

All planning algorithms presented in this work share a common incremental encoding using the *incmode* feature of *clingo* (Gebser et al. 2014). The program is divided into three subprograms: *base*, *step(t)* and *check(t)*.

The ***base*** subprogram (Line 2) defines the initial state s_0 , grounding the fluent values at time step 0. The predicate $holds(X, V, t)$ is used to represent fluents $X \in \mathcal{F}$ that have value $V \in \text{dom}(X)$ at timestep t .

The ***check(t)*** subprogram (Line 4) is a parametrized subprogram starting at $t = 0$. The integrity constraint checks if the goal condition is met, i.e. $s_*(X) = s_t(X)$ for each $X \in \text{dom}(s_*)$. The external atom $query(t)$ ensures this check is only active for the current search horizon t .

The ***step(t)*** subprogram (Line 6-9) is grounded for each timestep t starting at $t = 1$. Note that postconditions of applied actions in timestep t become effective in t , while their preconditions have to hold in $t - 1$.

- Line 6 generates the state candidates for timestep t , assigning every fluent X exactly one value V from its domain.
- Line 7 generates arbitrary sets of actions $occurs(A, t)$ for the current timestep. These actions and the generated state must obey the constraints in Lines 8 and 9.
- Line 8 ensures that the postcondition of an occurring action is actually applied in the state. This also asserts the confluence of the set of actions: if the postconditions of two actions have different values for the same fluent ($V = a^e(X) \neq a'^e(X) = V'$), the uniqueness of the value in the choice rule (Line 6) would be violated.
- Line 9 guarantees that fluents not affected by any action keep their value from the previous timestep. It rejects any change in a fluent's value that is not explained by an action.

In summary, Listing 2 establishes the state transitions consistent with the postcondition of the chosen actions. However, it does not yet verify the *applicability* of these actions regarding their preconditions, nor does it enforce the specific serializability requirements (e.g., acyclicity) of the

parallel semantics.

2.2.1 Extension for sequential plans

To restrict the generic transition system to valid *sequential plans*, we need to ensure that preconditions are satisfied and that each execution is restricted to a single action at a time. This is achieved by the extension shown in Listing 3.

```

1 :- occurs(A, t), prec(A, X, V), not holds(X, V, t-1).
2 :- #count{A : occurs(A, t)} > 1.
```

Listing 3: Extension of Listing 2 for encoding sequential plans

Line 2 implements the applicability check: it enforces that for any action occurring at timestep t , all its preconditions must hold in the preceding state s_{t-1} . Line 3 ensures that only one action can occur in each timestep t .

Example 5. Running this with our example, we get three possible sequential plans.

```

Answer: 1
occurs(getKey,1) occurs(lightOut,2) occurs(openDoor,3) occurs(goOut,4) occurs(closeDoor,5)
Answer: 2
occurs(getKey,1) occurs(openDoor,2) occurs(lightOut,3) occurs(goOut,4) occurs(closeDoor,5)
Answer: 3
occurs(lightOut,1) occurs(getKey,2) occurs(openDoor,3) occurs(goOut,4) occurs(closeDoor,5)
```

Listing 4: Output from clingo for sequential encoding

2.2.2 Extension for $\forall$ -step plans

```

1 :- occurs(A, t), prec(A, X, V), not holds(X, V, t-1).
2 :- occurs(A, t), prec(A, X, V), not post(A, X, _), not holds(X, V, t).
3 single(X, t) :- occurs(A, t), prec(A, X, V1), post(A, X, V2), V1 != V2.
4 :- single(X, t), #count{A : occurs(A, t), post(A, X, V)} > 1.
```

Listing 5: Extension of Listing 2 for encoding $\forall$ -step plans

The extension for $\forall$ -step plans is shown in Listing 5. Line 1 is identical to the sequential encoding, ensuring that the preconditions of all chosen actions are satisfied in the previous state s_{t-1} . To create a valid $\forall$ -step plan, we must ensure that each set of actions is executable in *any* order. Since Line 8 of the common core (Listing 2) already enforces confluence, the remaining requirement is that no action is allowed to invalidate the preconditions of another action in the same set. We distinguish two types of interactions between an action a and a fluent X in its precondition ($\text{dom}(a^c)$):

1. The action has X in its preconditions fluent X in its precondition but does not change it, (i.e. $X \in \text{dom}(a^c) \cap \text{dom}(a^e)$ and $a^c(X) = a^e(X)$ or $X \in \text{dom}(a^c)$ and $X \notin \text{dom}(a^e)$).
2. The action has fluent X in its preconditions and changes its value (i.e., $X \in \text{dom}(a^e)$ and $a^c(X) \neq a^e(X)$).

Line 2 handles case 1 by ensuring that X has to remained unchanged. However, case 2 requires strict isolation of these kinds of actions. If an action a invalidates its own precondition, no other action a' in the same timestep is allowed to modify X . If such an action a' existed, the execution order $\langle a', a \rangle$ would fail because a' would change X , invalidating the precondition of a . To enforce this, Line 3 marks such actions using the predicate `single(X, t)`. The integrity constraint in Line 4 then ensures that for any fluent X , at most one such action occurs per time step, preventing any interference.

Example 6. Running this with our example, we get two possible $\forall$ -step plans.

```

Answer: 1
occurs(getKey,1) occurs(openDoor,2) occurs(lightOut,2) occurs(goOut,3) occurs(closeDoor,4)
Answer: 2
occurs(lightOut,1) occurs(getKey,1) occurs(openDoor,2) occurs(goOut,3) occurs(closeDoor,4)

```

Listing 6: Output from clingo for $\forall$ -step encoding

2.2.3 encoding for $\exists$ -step plans

```

1 :- occurs(A, t) , prec(A, X, V), not holds(X, V ,t -1).
2 apply(A1, t) :- action(A1),
3     ready(A2, t) : post(A1, X, V1), prec(A2, X, V2), A1 != A2, V1 != V2.
4 ready(A, t) :- action(A), not occurs(A, t).
5 ready(A, t) :- apply(A, t).
6 :- action(A) , not ready(A, t).

```

Listing 7: Extension for $\exists$ -step plans

```

1 :- occurs(A, t), prec(A ,X ,V), not holds(X, V, t-1).
2 #edge((A1, t), (A2, t)): occurs(A1, t),
3     post(A1, X, V1), prec(A2, X, V2), A1 != A2, V1 != V2.

```

Listing 8: Alternative Extension for $\exists$ -step plans

There are two different ways to encode the extension for $\exists$ -step plans as seen in Listing 7 and Listing 8. As before, Line 1 in Listing 7 and 8 is the same as in the sequential and $\forall$ -step encoding. To ensure that there is at least one valid permutation, either an action a_1 that invalidates the precondition of another action a_2 has to be applied after a_2 or action a_2 does not occur in this timestep. For this, all actions has to be marked as *ready* (Line 6). An action is *ready* if it either does not occur in this timestep (Line 4) or if it is captured by *apply* (Line 2). An action can be safely applied, if all other actions whose preconditions it invalidates are ready, i.e. are either applied beforehand or not occurring in this timestep. Finally, Line 6 ensures that actions does not circularly invalidates each other.

Example 7. Running this with our example, we get three possible $\exists$ -step plans.

```

Answer: 1
occurs(getKey,1) occurs(openDoor,2) occurs(closeDoor,3) occurs(goOut,3) occurs(lightOut,3)
Answer: 2
occurs(getKey,1) occurs(openDoor,2) occurs(lightOut,2) occurs(closeDoor,3) occurs(goOut,3)
Answer: 3
occurs(lightOut,1) occurs(getKey,1) occurs(openDoor,2) occurs(closeDoor,3) occurs(goOut,3)

```

Listing 9: Output from clingo for $\exists$ -step encoding

2.2.4 encoding for relaxed $\exists$ -step plans

```

1 reach(X, V, t) :- holds(X, V, t - 1).
2 reach(X, V, t) :- occurs(A, t), apply(A, t), post(A, X, V).
3 apply(A1, t) :- action(A1), reach(X, V, t) : prec(A1, X, V);
4   ready(A2, t) : post(A1, X, V1), prec(A2, X, V2), A1 != A2 , V1 != V2.
5 ready(A, t) :- action(A), not occurs(A, t).
6 ready(A, t) :- apply(A, t).
7 :- action(A), not ready(A, t).

```

Listing 10: Extension for relaxed $\exists$ -step plans

Finally, the extension for relaxed $\exists$ -step plans as seen in Listing 10 is quite similar to the $\exists$ -step encoding. The difference between those two is that the precondition of all actions do not have to hold simultaneously but they can "help" each other out. This is done by defining the predicate `reach/3` in Line 1 and 2 as all fluents that can be reached by the step before or applied action in this timestep. An action can then be safely applied, if its precondition is reachable and as before, all actions that it invalidates are ready (Line 4). Again Line 7 ensures that applied action do not circularly invalidate each other.

Example 8. Running this with our example, we get three possible relaxed $\exists$ -step plans.

```

Answer: 1
occurs(openDoor,1) occurs(getKey,1) occurs(closeDoor,2) occurs(goOut,2) occurs(lightOut,2)
Answer: 2
occurs(openDoor,1) occurs(lightOut,1) occurs(getKey,1) occurs(closeDoor,2) occurs(goOut,2)
Answer: 3
occurs(openDoor,1) occurs(goOut,1) occurs(lightOut,1) occurs(getKey,1) occurs(closeDoor,2)

```

Listing 11: Output from clingo for relaxed $\exists$ -step encoding

2.3 Proof of Correctness

In order to prove the correctness of the presented encodings, DIMOPOULOS et al. 2019 defines the following:

- B is the rule in Line 1 of Listing 2.
- $Q(t)$ is the integrity constraint in the check program directive (Line 5) of Listing 2.
- $S(t)$ are the rules and integrity constraint in the step program directive (Lines 6-9) of Listing 2.
- $S^p(t)$ is the extension for the specific plans, where $p = s$ for Listing 3, $p = \forall$ for Listing 5, $p = \exists$ for Listing 7, $p = E$ for Listing 8 and $p = R$ for Listing 10.
- I is the set of facts representing a planning task $\langle \mathcal{F}, s_0, s_*, \mathcal{O} \rangle$.
- $\langle a_1, \dots, a_n \rangle$ is a sequence of actions.
- $\langle A_1, \dots, A_m \rangle$ is a sequence of sets of actions.
- $P^p(n) := I \cup B \cup Q(0) \cup \bigcup_{t=1}^n (Q(t) \cup S(t) \cup S^p(t)) \cup \{\text{query}(n).\}$, where $p \in \{s, \forall, \exists, E, R\}$.

Theorem 1 (Correctness of sequential plans (DIMOPOULOS et al. 2019)).

$\langle a_1, \dots, a_n \rangle$ is a sequential plan $\iff P^s(n)$ has a stable model M , s.t.

$\{\langle a, t \rangle \mid \text{occurs}(a, t) \in M\} = \{\langle a_t, t \rangle \mid 1 \leq t \leq n\}$.

Proof. We prove both directions separately.

($\Rightarrow$): Assume $A_n := \langle a_1, \dots, a_n \rangle$ is a sequential plan.

We construct a candidate set M based on the plan execution and show it is indeed a stable model of $P^s(n)$. The rule in B ensures that $\text{holds}(X, V, 0) = s_0(X)$ for each $X \in \mathcal{F}$. The choice rule (Line 5 of Listing 3) enforces that for any $0 \leq t \leq n$ and $X \in \mathcal{F}$, exactly one value V is assigned, i.e., $|\{V \mid \text{holds}(X, V, t) \in M\}| = 1$. Moreover, the integrity constraints (Lines 8 and 9) in $S(t)$ dictate the transition logic between states. For any

$1 \leq t \leq n$, the value V of a fluent X is determined by:

$$\text{holds}(X, V, t) \leftrightarrow \begin{cases} V = a_t^e(X) & \text{if } X \in \text{dom}(a_t^e) \\ V = s_{t-1}(X) & \text{otherwise} \end{cases}$$

Therefore, the set of **holds**-atoms in any stable model must satisfy:

$$\{\langle X, V, t \rangle \mid \text{holds}(X, V, t) \in M\} = \{\langle X, V, t \rangle \mid X \in \mathcal{F}, V = o(A_n, s_0)(X), 0 \leq t \leq n\} \quad (1)$$

Since the action sequence is fixed by the plan assumption ($\{\langle a, t \rangle \mid \text{occurs}(a, t) \in M\} = \{\langle a_t, t \rangle \mid 1 \leq t \leq n\}$), the only candidate for a stable model $P^s(n)$ is:

$$\begin{aligned} M = I \cup \{ & \text{occurs}(a_t, t) \mid 1 \leq t \leq n \} \\ & \cup \{ \text{holds}(X, V, t) \mid (X, V, t) \text{ satisfies (1)} \} \cup \{ \text{query}(n) \}. \end{aligned}$$

By the definition of a sequential plan, $o(a_t, s_{t-1})$ is defined for all $1 \leq t \leq n$, and the goal condition $s_n(X) = s_*$ for each $X \in \text{dom}(s_*)$ is met. Thus, the integrity constraint regarding preconditions (Line 1, Listing 3) and the goal check (Line 4, Listing 2) are satisfied. Since M is a set satisfying all rules and constraints, it is indeed a stable model of $P^s(n)$.

($\Leftarrow$): Assume that M is a stable model of $P^s(n)$ s.t. $\{\langle a, t \rangle \mid \text{occurs}(a, t) \in M\} = \{\langle a_t, t \rangle \mid 1 \leq t \leq n\}$.

We prove by induction over t , that for each $1 \leq t \leq n$, the following holds:

$$\{\langle X, V \rangle \mid \text{holds}(X, V, t) \in M\} = \{\langle X, V \rangle \mid X \in \mathcal{F}, V = o(\langle a_1, \dots, a_t \rangle, s_0)(X)\} \quad (2)$$

Base case $t = 0$:

The rule in B directly implies that $\{\langle X, V \rangle \mid \text{holds}(X, V, 0) \in M\} = \{\langle X, V \rangle \mid X \in \mathcal{F}, s_0(X) = V\}$, satisfying the equation for $t = 0$.

Induction step $t - 1 \mapsto t$:

Assume the hypothesis holds for $t - 1$, i.e., the state at $t - 1$ corresponds to $s_{t-1} = o(\langle a_1, \dots, a_{t-1} \rangle, s_0)$. The integrity constraint in Line 1 of Listing 3 in $S^s(t)$ ensures that the successor state $s_t = o(a_t, o(\langle a_1, \dots, a_{t-1} \rangle, s_0))$ is defined and the choice rule in Line 6 of Listing 2, together with the integrity constraints in $S(t)$ guarantee that

$$\{\langle X, V \rangle \mid \text{holds}(X, V, t) \in M\} = \{\langle X, V \rangle \mid X \in \mathcal{F}, V = o(\langle a_1, \dots, a_t \rangle, s_0)(X)\}.$$

Thus (2) holds for each $1 \leq t \leq n$. Given **query**(**n**), the integrity constraint in $Q(n)$ yields that $o(\langle a_1, \dots, a_n \rangle, s_0) = s_*(X)$ for each $X \in \text{dom}(s_*)$ and therefore $\langle a_1, \dots, a_n \rangle$ is a sequential plan.

□

Theorem 2 (Correctness of parallel plans (DIMOPOULOS et al. 2019)).

$\langle A_1, \dots, A_m \rangle$ is a $\forall$ -/exists-/relaxed exists-step plan $\iff P^p(m)$ has a stable model M s.t., $\{\langle a, t \rangle \mid \text{occurs}(a, t) \in M\} = \{\langle a, t \rangle \mid a \in A_t, 1 \leq t \leq m\}$ with $p \in \{\forall, \exists, R\}$.

Proof. We prove both directions separately.

($\Rightarrow$): Assume $\mathcal{A} = \langle A_1, \dots, A_m \rangle$ is a $\forall$ -/ $\exists$ -/relaxed $\exists$ -step plan. For each $1 \leq t \leq m$, let s_t be the state defined by applying the set A_t to s_{t-1} . Due to the confluence of A_t , s_t is well-defined as:

$$s_t(X) = \begin{cases} a^e(X) & \text{for each } a \in A_t \text{ and } X \in \text{dom}(a^e) \\ s_{t-1}(X) & \text{otherwise (inertia)} \end{cases}$$

Analogous to the proof of Theorem 1, consider any stable model M of $P^p(m)$ such that the action occurrences match the plan, i.e. $\{\langle a, t \rangle \mid \text{occurs}(a, t) \in M\} = \{\langle a, t \rangle \mid a \in A_t, 1 \leq t \leq m\}$. The fact rule in B , together with the choice rule and integrity constraints in $S(t)$, dictates the values of all fluents based on these actions. Thus, the set of **holds**-atoms in M is determined as:

$$\{\langle X, V, t \rangle \mid \text{holds}(X, V, t) \in M\} = \{\langle X, V, t \rangle \mid X \in \mathcal{F}, V = s_t(X), 0 \leq t \leq m\}.$$

We now distinguish the different cases of parallel plans:

Case 1: $\mathcal{A}$ is a $\forall$ -step plan.

For $1 \leq t \leq m$, the atom **single**(X, t) defined in $S^\forall(t)$ (Line 2 of Listing 5) is derived iff there exists some action $a \in A_t$ that invalidates its own precondition, i.e., $X \in \text{dom}(a^c) \cap \text{dom}(a^e)$ and $a^c(X) \neq a^e(X)$. Hence, $P^\forall(m)$ cannot have any stable model M apart from

$$\begin{aligned} M = & I \cup \{\text{occurs}(a, t) \mid a \in A_t, 1 \leq t \leq m\} \\ & \cup \{\text{holds}(X, V, t) \mid X \in \mathcal{F}, V = s_t(X), 0 \leq t \leq m\} \\ & \cup \{\text{single}(X, t) \mid a \in A_t, X \in \text{dom}(a^c) \cap \text{dom}(a^e), a^c(X) \neq a^e(X), 1 \leq t \leq m\} \\ & \cup \{\text{query}(m).\} \end{aligned}$$

such that $\{\langle a, t \rangle \mid \text{occurs}(a, t) \in M\} = \{\langle a, t \rangle \mid a \in A_t, 1 \leq t \leq m\}$.

It remains to show that M satisfies the integrity constraints in $S^\forall(t)$. Since for any $1 \leq t \leq m$, the successor state $s_t = o(\langle a_1, \dots, a_k \rangle, s_{t-1})$ is defined for *any* sequence $\langle a_1, \dots, a_k \rangle$ with $\{a_1, \dots, a_k\} = A_t$, no action can interfere with the precondition of

another action. Consequently, if $\text{single}(X, t)$ is derived (meaning an action invalidates its own precondition on X), no other action can modify X , as this would create a conflict in at least one permutation. Therefore, the integrity constraint restricting the count of actions affecting X alongside $\text{single}(X, t)$ is satisfied. Analogously, the constraint requiring preconditions to remain stable (Line 1 of Listing 5) is satisfied since $s_{t-1}(X) = a^c(X)$ for each $a \in A_t$, $X \in \text{dom}(a^c)$ and $1 \leq t \leq m$. One can check that the remaining rules and integrity constraints in $S(t)$ are satisfied as well, so M is a stable model of $P^\forall(m)$.

Case 2: $\mathcal{A}$ is a $\exists$ -step plan.

We analyze the encodings $P^\exists$ and P^E separately, starting with the acyclicity-based encoding P^E .

Analysis of P^E :

By definition of an $\exists$ -step plan, for every timestep $1 \leq t \leq m$, there is a sequence of actions $\langle a_1, \dots, a_k \rangle$ with $\{a_1, \dots, a_k\} = A_t$. A valid serialization requires that no action invalidates the precondition of a subsequent action. Formally, let $G_t = (A_t, E_t)$ be the conflict graph for timestep t , where an edge represents a destroyed precondition:

$$E_t = \{(a, a') \in A_t \times A_t \mid a \neq a', \exists X \in \text{dom}(a^e) \cap \text{dom}(a'^c) : a^e(X) \neq a'^c(X)\}.$$

Since a valid serialization exists, $\langle a_1, \dots, a_k \rangle$ constitutes a topological ordering of G_t , i.e. there exists a sequence of applied actions. A directed graph possesses a topological ordering iff it is a Directed Acyclic Graph (DAG). Thus, G_t must be acyclic.

Now consider the candidate model M constructed as in Case 1:

$$M = I \cup \{\text{occurs}(a, t) \mid a \in A_t, 1 \leq t \leq m\} \\ \cup \{\text{holds}(X, V, t) \mid X \in \mathcal{F}, V = s_t(X), 0 \leq t \leq m\} \cup \{\text{query}(m)\}.$$

The `#edge` directive in Listing 8 (Lines 2-3) generates an edge between two occurring actions (a, t) and (a', t) exactly if the condition for E_t is met. Consequently, the set of edges declared in the ASP model corresponds exactly to the union of all conflict graphs $\bigcup_{t=1}^m E_t$. Since each G_t is acyclic and the graphs for different timesteps $1 \leq t < t' \leq m$ are vertex-disjoint ($(a, t) \neq (a, t')$), the overall graph induced by M is acyclic. Therefore, the acyclicity constraint is satisfied, and M is a stable model of $P^E(m)$.

Analysis of $P^\exists$:

The rules defining `apply` and `ready` in $S^\exists(t)$ (Lines 2-5 in Listing 7) effectively traverse the dependency graph G_t . Since G_t is acyclic, there exists a topological ordering of the actions. The derivation of atoms proceeds along this ordering: Actions with no dependencies (sinks in G_t) are derived as `apply` first, making them `ready`. This, in turn, satisfies the conditions for their predecessors. Because G_t is finite and acyclic, this inductive process

reaches a fixpoint where $\mathbf{apply}(a, t)$ and $\mathbf{ready}(a, t)$ hold for all $a \in \mathcal{O}$ and $1 \leq t \leq m$. Consequently, the integrity constraint requiring all actions to be ready (Line 6 in Listing 7) is satisfied. Thus, $P^\exists(m)$ extends the base model M to:

$$M' = M \cup \{\mathbf{apply}(a, t) \mid a \in \mathcal{O}, 1 \leq t \leq m\} \cup \{\mathbf{ready}(a, t) \mid a \in \mathcal{O}, 1 \leq t \leq m\}$$

such that $\{\langle a, t \rangle \mid \mathbf{occurs}(a, t) \in M'\} = \{\langle a, t \rangle \mid a \in A_t, 1 \leq t \leq m\}$. As established previously, the state transition rules are satisfied by M (and thus by M'). Since the additional constraints in $P^\exists$ regarding circularity are also satisfied by the completeness of the $\mathbf{ready}$ derivation, M' is indeed a stable model of $P^\exists(m)$.

Case 3: $\mathcal{A}$ is a *relaxed* $\exists$ -step plan.

For each $1 \leq t \leq m$, by definition of a relaxed $\exists$ -step plan, there exists a serialization $\langle a_1, \dots, a_k \rangle$ of A_t such that the sequential execution is valid. This means $o(a_i, o(\langle a_1, \dots, a_{i-1} \rangle, s_{t-1}))$ is defined for all $1 \leq i \leq k$.

Consequently, the preconditions of any action a_i are satisfied by the state resulting from the application of its predecessors in $\langle a_1, \dots, a_n \rangle$. The rules in $S^R(t)$ (Listing 10) simulate this reachability:

- (i) The atom $\mathbf{reach}_>(X, V, t)$ is derived for all values V that are either present in s_{t-1} or produced by an applied action.
- (ii) The atom $\mathbf{apply}(a, t)$ is derived for any action $a \in \mathcal{O}$ whose preconditions are satisfied by the reachable values.
- (iii) The atom $\mathbf{ready}(a, t)$ is derived for all $a \in \mathcal{O}$ (either because they are applied, or because they do not occur).

Since the serialization $\langle a_1, \dots, a_k \rangle$ is valid, the least fixpoint of these rules guarantees that $\mathbf{apply}(a, t)$ is derived for all $a \in A_t$.

Thus, the stable model M of $P^R(m)$ is:

$$\begin{aligned} M = & I \cup \{\mathbf{occurs}(a, t) \mid a \in A_t, 1 \leq t \leq m\} \\ & \cup \{\mathbf{holds}(X, V, t) \mid X \in \mathcal{F}, V = s_t(X), 0 \leq t \leq m\} \\ & \cup \{\mathbf{reach}(X, V, t) \mid X \in \mathcal{F}, V \in \{s_{t-1}, s_t\}, 1 \leq t \leq m\} \\ & \cup \{\mathbf{apply}(a, t) \mid a \in \mathcal{O} : a^c(X) \in \{s_{t-1}(X), s_t(X)\} \text{ for each } X \in \text{dom}(a^c)\} \\ & \cup \{\mathbf{ready}(a, t) \mid a \in \mathcal{O}, 1 \leq t \leq m\} \\ & \cup \{\mathbf{query}(m)\} \end{aligned}$$

Since M contains $\mathbf{ready}(a, t)$ for all actions, the integrity constraint in Line 7 of Listing 10 is satisfied. Furthermore, all state transition constraints are met by the definition of s_t . Therefore, M is a stable model of $P^R(m)$.

($\Leftarrow$): Assume that M is a stable model of $P^p(m)$ such that $\{\langle a, t \rangle \mid \mathbf{occurs}(a, t) \in M\} =$

$\{\langle a, t \rangle \mid a \in A_t, 1 \leq t \leq m\}$.

For any $1 \leq t \leq m$, the choice rule in Line 6 of Listing 2 ensures that the set of **holds**-atoms induces a state function s_t defined by $s_t(X) = V \Leftrightarrow \text{holds}(X, V, t) \in M$.

Moreover, the rules and integrity constraints in $S(t)$ dictate the state transition. Specifically, the integrity constraint in Line 8 of Listing 2 ensure that for each $a \in A_t$ and $X \in \text{dom}(a^e)$, $s_t(X) = a^e(X)$ holds. Since M is a stable model, contradicting effects on the same variable are impossible (as they would violate the constraints), which implies that A_t is confluent. For unaffected fluents, the integrity constraint in Line 9 of Listing 2 yield $s_t(X) = s_{t-1}(X)$ for each $X \in \mathcal{F} \setminus \bigcup_{a \in A_t} \text{dom}(a^e)$.

Finally, given the fact **query**(m), the integrity constraint in $Q(m)$ ensures that $s_m(X) = s_*(X)$ for all $X \in \text{dom}(s_*)$, satisfying the goal condition.

We now distinguish the different cases to show that A_t is $\forall$ -/ $\exists$ -/*relaxed* $\exists$ -step serializable in s_{t-1} based on the specific constraints of each encoding.

Case 1: To show: A_t is $\forall$ -step serializable.

Let $\langle a_1, \dots, a_k \rangle$ be an arbitrary sequence of actions such that $\{a_1, \dots, a_k\} = A_t$. We prove by induction that for all $1 \leq i \leq k$, the transition $o(\langle a_1, \dots, a_i \rangle, s_{t-1})$ is defined.

Base case: $i = 1$:

The integrity constraint checking preconditions (Line 1 of Listing 5) guarantees that a_1 is applicable in s_{t-1} , i.e., $o(a_1, s_{t-1})$ is defined.

Inductive step $i \mapsto i + 1$:

Assume $o(\langle a_1, \dots, a_i \rangle, s_{t-1})$ is defined. We must show that a_{i+1} is applicable in the resulting intermediate state $s_{temp} = o(\langle a_1, \dots, a_i \rangle, s_{t-1})$. Consider any fluent X in the precondition of a_{i+1} . If $X \notin \bigcup_{1 \leq j \leq i} \text{dom}(a_j^e)$, then no preceding action has modified X . Thus $s_{temp}(X) = s_{t-1}(X)$, and the precondition holds trivially because a_{i+1} is applicable in s_{t-1} . Therefore, we focus on the critical case where $X \in \text{dom}(a_{i+1}^e) \cap \bigcup_{1 \leq j \leq i} \text{dom}(a_j^e)$. This implies there exists a preceding action a_j that has a postcondition on X . We distinguish two cases based on the effect of a_{i+1} itself:

(i) $X \notin \text{dom}(a_{i+1}^e)$:

The integrity constraint in Line 2 of Listing 5 requires that if a_{i+1} has a precondition on X but no postcondition, the value of X cannot change in the final state s_t . Since a_j writes X , we have $s_t(X) = a_j^e(X)$. The constraint implies $s_{t-1}(X) = s_t(X)$, meaning a_j effectively did not change the value relative to s_{t-1} . Thus $s_{temp}(X) = a_{i+1}^e(X)$.

(ii) $X \in \text{dom}(a_{i+1}^e)$:

The integrity constraint in Line 4 of Listing 5 ensures that if an action (a_{i+1}) has a precondition on X and sets a conflicting postcondition ($a^e(X) \neq a^e(X)$), it is marked as **single**. However, since another action a_j also acts on X (as established by the intersection), the integrity constraint in Line 5 would be violated. Since

M is a stable model, this implies that $\text{single}(X, t) \notin M$ and $s_t(X) = a_{i+1}^e(X) = a_{i+1}^c(X) = s_{t-1}(X)$ must be the case.

In both cases, $s_{temp}(X) = a_{i+1}^c(X)$, so $o(\langle a_1, \dots, a_{i+1} \rangle, s_{t-1})$ is defined. Therefore, by induction, the entire sequence is valid, and A_t is $\forall$ -step serializable.

Case 2: To show: A_t is $\exists$ -step serializable.

The integrity constraint in Line 1 of Listing 7 guarantees that every action $a \in A_t$ is applicable in s_{t-1} , i.e., $o(a, s_{t-1})$ is defined. Furthermore, the integrity constraint in Line 6 ensures that $\text{ready}(a, t) \in M$ for each $a \in \mathcal{O}$ and the rules in Lines 4-5 that $\text{apply}(a, t) \in M$ for all $a \in A_t$. The rule defining **apply** (Lines 2-3) imposes a dependency: an action a can only be applied if all actions a' whose preconditions it invalidates are already **ready**. Since M is a stable model, there exists a derivation sequence of the **apply** atoms that respects these dependencies. Let $\langle a_1, \dots, a_k \rangle$ be the sequence of actions in A_t ordered by their derivation step. Consider any action a_i in the sequence and a fluent X in the precondition. Suppose that a preceding action a_j ($j < i$) invalidates this precondition, i.e., $a_j^e(X) \neq a_i^c(X)$. By the definition of **apply** in the Lines 2-3, a_j would have to wait for a_i to become **ready** before it could be applied. This would imply that a_i is derived before a_j , contradicting $j < i$. Therefore, for all $j < i$, a_j does not invalidate a_i . Thus, $s_{t-1}(X)$ is either preserved ($s_{t-1}(X) = s_t(X)$) or modified confluent ($a_j^e(X) = a_i^c(X)$), ensuring $o(\langle a_1, \dots, a_k \rangle, s_{t-1})$ is defined.

Regarding the alternative encoding using the **#edge** directive (Listing 8), the graph logic ensures serializability explicitly. For any $1 \leq t \leq m$, the directive defines the conflict graph $G_t = (A_t, E_t)$ where a directed edge $(a, a') \in E_t$ exists iff action a invalidates a precondition of action a' . Since $P^E(m)$ enforces that G_t is acyclic, there exists a sequence $\langle a_1, \dots, a_k \rangle$ comprising the actions in A_t such that no edge connects any predecessor to a successor. Formally, for any a_i in the sequence, there is no a_j with $j < i$ such that $(a_j, a_i) \in E_t$. This absence of destructive edges from the past implies that for any X in the precondition, either no predecessor changes X or it is modified consistently, i.e. $s_{t-1}(X) = a_i^c(X)$ or $a_j^e(X) = a_i^c(X)$. Thus, $o(\langle a_1, \dots, a_i \rangle, s_{t-1})$ is defined for all i , and in particular $o(\langle a_1, \dots, a_k \rangle, s_{t-1})$ is defined, verifying that A_t is $\exists$ -step serializable.

Case 3: To show: A_t is *relaxed* $\exists$ -step serializable.

In view of the rule in Line 1 of Listing 10, a fluent value is reachable if $s_{t-1}(X) = V$, i.e., $\text{reach}(X, V, t) \in M$. The recursive structure of the rules for **apply** (Lines 3-4) and **reach** (Line 2) imposes a dependency ordering: An action a is only derived as **apply**(a,t) if all its preconditions are already in the extension of the **reach** predicate.

Since M is a stable model, there exists a finite derivation sequence for all atoms in

M. Let $\langle a_1, \dots, a_k \rangle$ be the sequence of actions in A_t ordered by the step in which their corresponding **apply** atom was derived (or satisfied). We prove that this sequence is a valid serialization.

Consider any action a_i in this sequence and any fluent X in the precondition. The derivation of **apply**(a_i, t) requires that **reach**($X, a_i^c(X), t$) was previously derived. By the definition of the **reach** rules, this value must originate either from:

- (i) The state s_{t-1} (Line 1), which implies $s_{t-1}(X) = a_i^c(X)$, or
- (ii) The effect of a previously applied action a_j (Line 2), which implies $a_j \in \{a_1, \dots, a_{i-1}\}$ and $a_j^e(X) = a_i^c(X)$.

In both cases, the precondition of a_i is satisfied by the state constructed from the initial state s_{t-1} and the effects of the predecessors $\langle a_1, \dots, a_{i-1} \rangle$. Thus, $o(\langle a_1, \dots, a_k \rangle, s_{t-1})$ is defined.

□

3 Method

3.1 Pipeline Overview

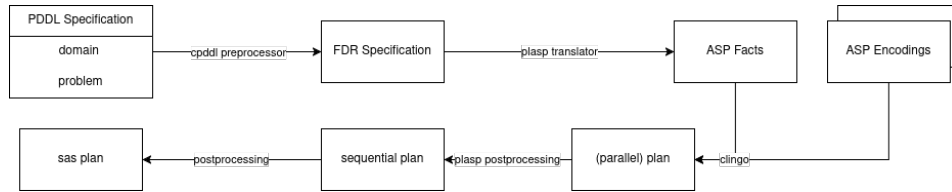


Figure 3.1: Pipeline for testing the different planning encodings

To test and compare the different parallel planning encodings against each other and against the sequential planning encoding, we set up a python pipeline using the lab framework (Seipp et al. 2017) and the unofficial collection of (sequential) IPC benchmark instances from the downward project ¹.

Starting at a planning task in the Planning Domain Definition Language (PDDL), we first preprocess it using cpddl (Fišer 2025) that converts it into a finite domain representation (FDR). Plasp then takes the FDR as input and translates it into ASP facts representing the planning task. Further information of this step can be found in (DIMOPOULOS et al. 2019). Using clingo with a modified version of the incremental encodings presented above together with the facts from the output of plasp, we can then compute sequential, $\forall$ -step, $\exists$ -step or relaxed $\exists$ -step plans as stable models. Finally, to validate the given plan, we postprocess the parallel representation back into a sequential plan, using the postprocess encoding provided by plasp and validate the plan with VAL ². Each step has some optimizations and configurations that will be explained in the following sections.

3.2 Changes in the ASP encodings

To use the presented encodings in our pipeline, we had to make some small changes to the original encodings from (DIMOPOULOS et al. 2019).

¹<https://github.com/aibasel/downward-benchmarks>

²<https://github.com/KCL-Planning/VAL>

```

1  #include <incmode>.
2  #program base.
3  fluent(X) :- variable(X).
4  fluent(X,V) :- initialState(X,V), fluent(X).
5  value(X,V) :- contains(X, V).
6  prec(A, X, V) :- precondition(A, X, V).
7  post(A, X, V) :- postcondition(A, _, X, V).
8  ...
9  :- fluent(X), #count{V : holds(X,V,0)} > 1.
10 ...
11 #program step(t).
12 ...
13 #show.
14 #show occurs(A, t): occurs(A, t).

```

Line 3-6 just renames the predicates from plasp's output to match the ones used in the encodings, while Line 7 uses the wildcard `_` to ignore the effect condition. This let clingo assume that all effects are fulfilled, which reduces the search space but might lead to not fully optimal plans. Line 9 ensures that each fluent has at most one value in the initial state and finally Line 13-14 specify which atoms should be shown in the output.

3.3 Optimizations

3.3.1 Mutexes

Definition 7 *mutex and mutex groups*. A set of facts $M \subseteq \mathcal{F}$ is called

- (i) a *mutex* $\iff$ for all reachable states $s : M \not\subseteq s$.
- (ii) a *mutex group* $\iff$ for all reachable states $s : |M \cap s| \leq 1$.

In other words, a mutex is a set of fluents that can never be true at the same time in any reachable state, while a mutex group is a set of fluents where at most one fluent can be true at the same time. Mutexes and mutex groups can be used to optimize the planning encodings by reducing the search space. For this, we extend our encoding in Listing 2 with the following rules:

```

1      #program base.
2      ...
3      mutex(G,X) :- mutexGroup(G), contains(G,X,V), fluent(X,V).
4      mutex(G)   :- mutexGroup(G), #count{X : mutex(G,X)} > 1.
5      :- mutex(G), #count{X,V : holds(X,V,0), contains(G,X,V)} > 1.
6      ...
7      #program step(t).
8      ...
9      :- mutex(G), #count{X,V : holds(X,V,t), contains(G,X,V)} > 1.
10     ...

```

Listing 12: Expansion for mutexes

Here, the mutex group G is given by plasp as facts of the form *mutexGroup*/1 and *contains*/3. In the base program, we first define which fluents belong to a mutex group (Line 3), where a group must contain at least two fluents (Line 4). The integrity constraint in Line 5 ensures that no more than one fluent of each mutex group holds in the initial state and the integrity constraint in Line 9 ensures the same for each state in timestep t .

3.3.2 Guess and Check

An optimization specific to the (relaxed) $\exists$ -step encodings is the guess and check approach. The general idea is to first guess a solution without looking for actions that circularly invalidate each other and afterwards check if the guessed solution has a cycle. Formally, we encode the problem by a pair of programs $\langle G, C \rangle$ where G is the guessing program and C is the checking program. Stable models M of G yields a solution if $C \cap M$ is *unsatisfiable*. For making "systematic" guesses, (DIMOPOULOS et al. 2019) proposes to use the core program ($S(t)$) in Listing 2 together with the integrity constraint in Line 1 of Listing 3, 5 or 7 (B) and the facts representing a planning task. Thus, a stable model M of G is s.t. the pre- and postconditons hold for all actions applied in parallel (Line 8 of Listing 2 and Line 1 of Listing 3, 5 or 7). and the integrity constraint in Line 8 of Listing 2 also ensures confluence of applied actions. The only thing remained to be checked is if actions circularly invalidate each other, i.e. $a^e(X) \neq \tilde{a}^e(X)$ and $\tilde{a}^e(\tilde{X}) \neq \tilde{a}^e(\tilde{X})$. This can be accomplished by taking the atoms over *occurs*/2 from M as facts together with a program C containing the facts representing a planning task and the following encoding:

```

1  #include <incmode>.
2  #program check(t).
3  :- query(t), not cycle(t).
4  #program step(t).
5  prec(A, X, V, t) :- prec(A, X, V), occurs(A, t).
6  post(A, X, V, t) :- post(A, X, V), occurs(A, t).
7  action(A, t) :- occurs(A, t).
8  apply(A1, t) :- action(A1, t),
9      ready(A2, t) :- post(A1, X, V1, t), prec(A2, X, V2, t), A1 != A2 , V1 != V2.
10 ready(A, t) :- action(A, t), not occurs (A, t).
11 ready (A, t) :- apply(A, t).
12 cycle(t) :- action (A, t), not ready (A, t).

```

Listing 13: Encoding for checking circular invalidation

Here, the maximum value for t in subprogram *check* and *step* is $m = \max\{t \mid \text{occurs}(a, t)\}$ and C becomes *unsatisfiable* if $t > m$. Like in Listing 7, we check for each t if all actions $A \in \text{occurs}(A, t)$ can be applied by the predicate *ready* (Line 7-11). If actions in A circularly invalidate each other, *ready*(A, t) is underivable for these actions and thus *cycle*(t) (Line 12) in step t is derived. Hence, C becomes satisfiable if there is a circular invalidation in step t , which means that there is no permutation of the actions A , s.t. the successor state is defined and therefore not (relaxed) $\exists$ -step serializable.

3.3.3 Heuristic

The general idea of heuristics in clingo described by (DIMOPOULOS et al. 2019) is to extend the search heuristic by giving each atom a

- *level* ($\in \mathbb{Z}$, 0 by default), allowing to modify the order of which atoms are selected first
- *sign* ($\in \mathbb{Z}$, 0 by default, where values > 0 corresponds to *true*, < 0 to *false* and 0 to *undefined*), which allows for controlling the truth value assigned to atoms.

During solving, these values get modified during the activation of **#heuristic** directives. The search heuristic of clingo selects the atom with the highest level and sets it to its associated *sign*, if *sign* defined, else switching to the default *sign* heuristic.

```
1 #program step(t).  
2 #heuristic holds(X, V, t-1):    holds(X, V, t). [2147483647-t, true]  
3 #heuristic holds(X, V, t-1):    not holds(X, V, t). [2147483647-t, false]
```

Listing 14: backward propagating heuristic

The heuristic in Listing 14 aims starting at the goal of a planning task and then searching backwards. Here both lines have the level $l = 2^{31} - 1 - t$ where $2^{31} - 1$ is the largest integer supported by clingo, giving the starting steps the highest priority. The second line expresses that whenever clingo made $holds(X, V, t)$ true, $holds(X, V, t - 1)$ should be made true at the level l as well. Similarly, the directive in Line 3 is activated when an instance of $holds(X, V, t)$ becomes false, in which case $holds(X, V, t - 1)$ should be made false at level l as well. Both directives associates atoms over holds/3 with higher level the earlier the state is, i.e. the value for t is smaller, which intends to bias the search of *clingo* to establish the goal conditions as early as possible.

3.4 Experiment

To compare the different planning encodings and the effectiveness of the optimizations, we run four experiments with the pipeline described above. Each experiment run on the same set of benchmark instances with the sequential, $\forall$ -step, *relaxed* $\exists$ -step, *guess and check* and both versions of the $\exists$ -step encodings. The Guess and Check algorithm and the fallback described above are started one after the other with the time limit of 30 minutes, where the times are added together if the check fails and the $\forall$ -step encoding finds a solution. If the summed up time is below the time limit, the problem is considered solved.

Aborted runs are considered unsolved and assigned the time nan. *Cpddl* is equipped with and without the h2 option that extends the search for mutexes and the planning encodings with and without the backward propagating heuristic. The experiments were run on the bwiclust3.0 server of the Karlsruhe Institute of Technology (KIT) using nodes with Intel Xeon Gold 6248R CPUs (3.0 GHz, 24 cores). Each run was limited to one CPU core, where *clingo* and *cpddl* were given a time limit of 30 minutes and a memory limit of 32 GiB each.

4 Results

4.1 Overall Performance

To obtain a comprehensive overview of the performance of the proposed parallel encodings compared to the sequential baseline, we analyze the aggregated results across all benchmark domains. In this section, we focus on the baseline configurations without additional optimizations (such as mutex groups or heuristics) to establish a clear ground truth. We compare the Sequential encoding (S) against the $\forall$ -step ($\forall$), $\exists$ -step ($\exists$), the alternative $\exists$ -step (E), the *relaxed* $\exists$ -step (R), and the Guess-and-Check strategy (G&C).

Figure illustrates the coverage (number of solved instances) for each domain, split into two parts for readability. Figure displays the average runtime for instances that were successfully solved by all algorithm.

4.1.1 Coverage Analysis

The coverage results in Figure reveal several distinct trends regarding the effectiveness of parallel planning.

Sequential vs. Parallel: In domains that inherently allow for a high degree of concurrency, parallel encodings significantly outperform the sequential approach. Notable examples include *airport* and *elevators*. In *airport*, the sequential encoding (S) solves only 11 out of 50 instances, whereas the $\forall$ -step encoding (V) solves 49/50 and the $\exists$ -step variants solve 48/50. Similarly, in *floortile*, the sequential approach fails to solve any instance (0/40), while the standard $\exists$ -step encoding solves 37/40. This confirms that the reduced time horizon T of parallel plans effectively mitigates the search depth explosion found in sequential planning.

Comparison of Parallel Semantics: Comparing the different parallel semantics yields interesting insights into the trade-off between plan flexibility and encoding complexity:

- **$\forall$ -step (V):** Despite being the most restrictive parallel semantic, the $\forall$ -step encoding performs surprisingly well in many domains (e.g., *parcprinter*, *woodworking*). This suggests that many IPC domains do not require complex interleaved actions, and the simpler encoding of $\forall$ -step (avoiding quadratic constraints) puts less burden on the grounder and solver.

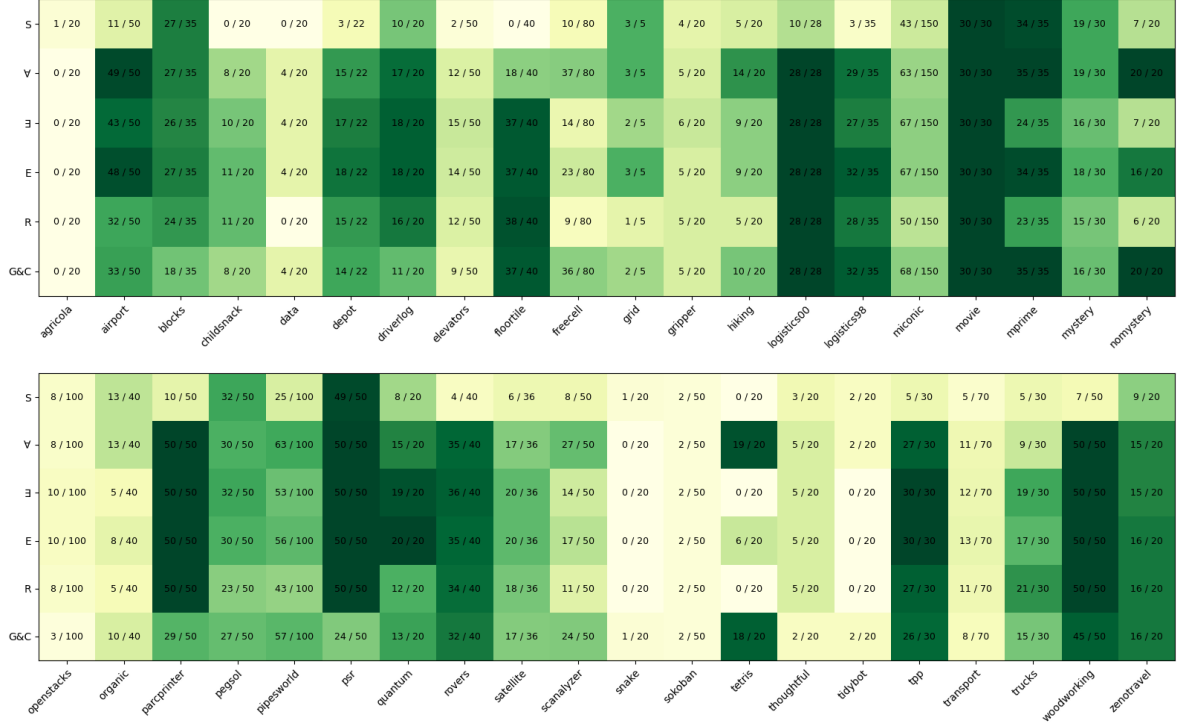


Figure 4.1: Coverage analysis: Number of solved instances per algorithm across benchmark domains. The heatmap indicates the absolute number of solved instances (left) out of the total available instances (right).

- **$\exists$ -step ($\exists$ / **E**):** The $\exists$ -step semantics show their strength in domains with complex dependencies. A prime example is *floortile*, where $\forall$ -step only solves 18/40 instances, but $\exists$ -step solves 37/40. This indicates that the flexibility of $\exists$ -step allows for finding plans in restricted environments where $\forall$ -step fails. The alternative encoding (**E**) generally performs on par with the standard $\exists$ -step, showing no significant deviation in coverage.
- **Relaxed $\exists$ -step (**R**):** The relaxed encoding typically solves fewer instances than the standard $\exists$ -step (e.g., *freecell*: 9 vs. 14). While theoretically allowing shorter plans, the relaxed encoding likely increases the search space or grounding size, leading to resource exhaustion on harder instances.

4.1.2 Runtime Analysis

The runtime heatmaps in Figure corroborate the coverage findings.

- ****Efficiency of Parallelism:**** In domains where parallel planning is superior in coverage, it is also drastically faster. In *airport*, the sequential solver averages 10.16s, while the $\forall$ -step solver requires only 0.13s. In *parcarprinter*, the difference is even more pronounced (191s vs 0.03s).

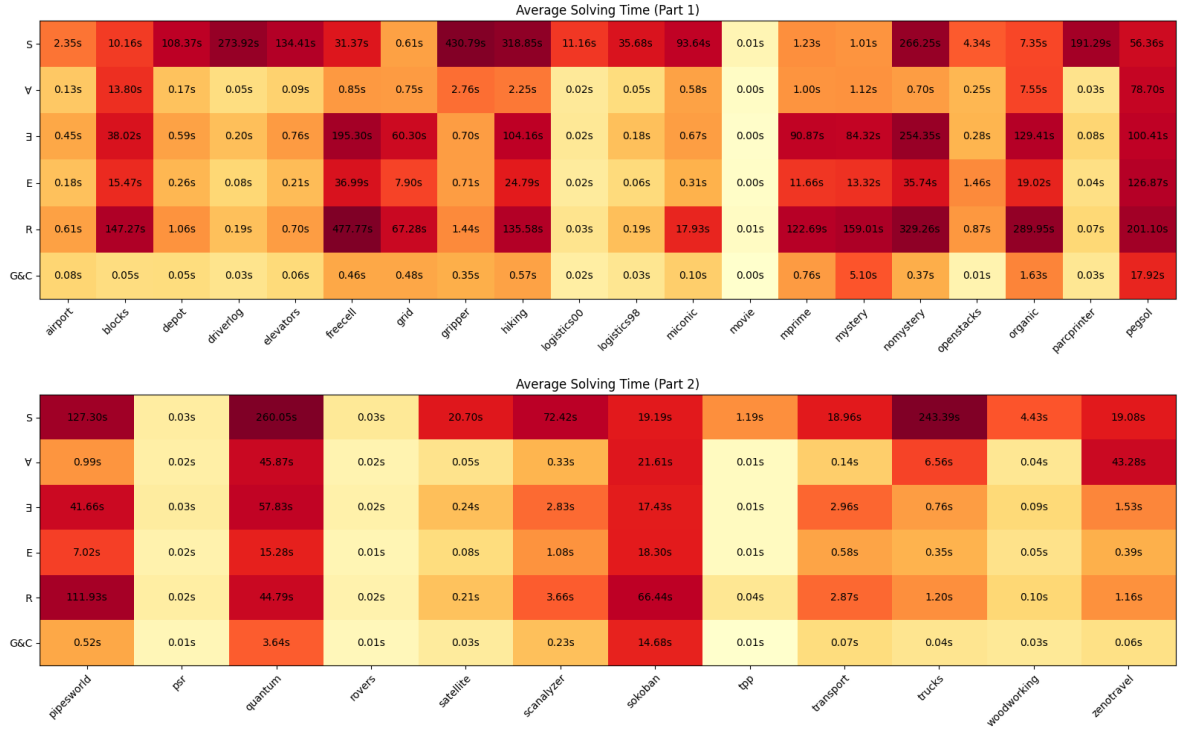


Figure 4.2: Runtime analysis: Average solving time (in seconds) for successfully solved instances. Red cells indicate higher runtimes, while light yellow cells indicate faster performance.

- ****Overhead of Complexity:**** In some domains, such as *sokoban*, the Relaxed encoding (R) shows a significantly higher average runtime (66.44s) compared to $\forall$ -step (21.61s) or even Sequential (19.19s). This highlights that the overhead of the more complex reachability predicates in the relaxed encoding can outweigh the benefit of fewer time steps if the domain does not strictly require such flexibility.
- ****Guess-and-Check Potential:**** The Guess-and-Check (G&C) strategy shows promising runtimes in specific domains. For example, in *airport*, it achieves the lowest average runtime of 0.08s. However, its performance varies, which will be discussed in detail in Section

Bibliography

- Gelfond, Michael and Vladimir Lifschitz (1988). “The Stable Model Semantics for Logic Programming”. In: *Proceedings of International Logic Programming Conference and Symposium*. Ed. by Robert Kowalski, Bowen, and Kenneth. MIT Press, pp. 1070–1080. URL: <http://www.cs.utexas.edu/users/ai-lab?gel88>.
- Dimopoulos, Yannis, Bernhard Nebel, and Jana Koehler (Oct. 1997). “Encoding Planning Problems in Nonmonotonic Logic Programs”. In: ISBN: 978-3-540-63912-1. DOI: [10.1007/3-540-63912-8_84](https://doi.org/10.1007/3-540-63912-8_84).
- Helmert, M. (2006). “The Fast Downward Planning System”. In: *Journal of Artificial Intelligence Research* 26, pp. 191–246.
- Rintanen, Jussi, Keijo Heljanko, and Ilkka Niemelä (2006). “Planning as satisfiability: parallel plans and algorithms for plan search”. In: *Artificial Intelligence* 170.12, pp. 1031–1080. ISSN: 0004-3702. DOI: [10.1016/j.artint.2006.08.002](https://doi.org/10.1016/j.artint.2006.08.002). URL: <https://www.sciencedirect.com/science/article/pii/S0004370206000774>.
- Wehrle, Martin and Jussi Rintanen (Dec. 2007). “Planning as Satisfiability with Relaxed $\exists$ -Step Plans.” In: vol. 4830, pp. 244–253. ISBN: 978-3-540-76926-2. DOI: [10.1007/978-3-540-76928-6_26](https://doi.org/10.1007/978-3-540-76928-6_26).
- Gebser, Martin et al. (May 2014). *Clingo = ASP + control: Preliminary report*. Tech. rep. 1405.3694. arXiv. URL: <https://arxiv.org/abs/1405.3694v1>.
- Seipp, Jendrik et al. (2017). *Downward Lab*. <https://doi.org/10.5281/zenodo.790461>.
- DIMOPOULOS, YANNIS et al. (Jan. 2019). “plasp 3: Towards Effective ASP Planning”. In: *Theory and Practice of Logic Programming* 19.3, pp. 477–504. ISSN: 1475-3081. DOI: [10.1017/s1471068418000583](https://doi.org/10.1017/s1471068418000583). URL: <http://dx.doi.org/10.1017/S1471068418000583>.
- Fišer, Daniel (Nov. 2025). *cpddl*. Version 1.5. DOI: [10.5281/zenodo.16423302](https://doi.org/10.5281/zenodo.16423302). URL: <https://doi.org/10.5281/zenodo.16423302>.